



# **Workpath SDK Accessory Overview**

**Product Group: HP Workpath SDK**

**Version: 1.6.3**

**Revised: November 11, 2025**

HP Inc.  
11311 Chinden Blvd.  
Boise, ID 83714 USA

HP Confidential

# Table of Contents

|          |                                                                |           |
|----------|----------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction .....</b>                                      | <b>3</b>  |
| 1.1      | Document conventions .....                                     | 3         |
| <b>2</b> | <b>Accessory .....</b>                                         | <b>4</b>  |
| 2.1      | Authentication Process Initiation with Accessory .....         | 4         |
| 2.1.1    | Non-UI Initiated Authentication .....                          | 4         |
| 2.2      | Accessory types for getting valid Accessory's context ID ..... | 4         |
| 2.2.1    | Shared accessories .....                                       | 4         |
| 2.2.2    | Owned accessories.....                                         | 4         |
| <b>3</b> | <b>Accessory flows .....</b>                                   | <b>5</b>  |
| 3.1      | Registration .....                                             | 5         |
| 3.2      | Obtain valid context ID.....                                   | 5         |
| 3.3      | Open.....                                                      | 5         |
| 3.4      | Get specific accessory information .....                       | 5         |
| 3.5      | Get accessory event when active context is changed.....        | 5         |
| 3.6      | Handle accessory reports.....                                  | 5         |
| 3.6.1    | Asynchronously read reports .....                              | 6         |
| 3.6.2    | Synchronously read reports .....                               | 6         |
| 3.6.3    | Write reports.....                                             | 6         |
| 3.7      | Close.....                                                     | 6         |
| 3.8      | Unregistration .....                                           | 6         |
| <b>4</b> | <b>Authentication Process with accessory.....</b>              | <b>7</b>  |
| <b>5</b> | <b>Create Accessory Agent for Authentication.....</b>          | <b>9</b>  |
| <b>6</b> | <b>Accessory Agent Sample on a device .....</b>                | <b>12</b> |
| 6.1      | Extends AbstractAuthenticationService .....                    | 13        |
| 6.2      | Extends BroadcastReceiver .....                                | 16        |
| 6.2.1    | HID accessory data report in a broadcast intent .....          | 16        |
| 6.2.2    | String-typed accessory data report in a broadcast intent ..... | 20        |
| 6.3      | Initiate authentication process.....                           | 24        |
| <b>7</b> | <b>Accessory Service .....</b>                                 | <b>25</b> |
| 7.1      | Accessory Service Implementation.....                          | 25        |
| 7.2      | Bind Service.....                                              | 25        |
| 7.3      | Alarm Settings .....                                           | 26        |
| 7.4      | onStart() .....                                                | 26        |
| 7.5      | Unbind Service .....                                           | 26        |
| 7.6      | Execute accessory service through AlarmManager .....           | 26        |
| <b>8</b> | <b>Legal notice .....</b>                                      | <b>27</b> |
| <b>9</b> | <b>Support.....</b>                                            | <b>28</b> |

# 1 Introduction

Accessory provides a use case scenario on how to make accessory agent for authenticating front panel users and receiving the events from accessory on Workpath enabled devices.

## 1.1 Document conventions

The following conventions are used throughout this document to define, describe, and discuss the API: function prototypes, data structure definitions, enumerated constants.

- **Name** – Bold signifies item names such as button names and menu items.
- <name> – Angle brackets mark URLs.
- {value} – Curly braces represent values to be replaced.
- Identifiers for defined constants use all UPPERCASE letters.
- *Name* – Italicized type refers to formal function parameters, data structure types and fields, and other defined or enumerated types in the text body (for example, *streamtype*).
- Numbers are in decimal format unless otherwise noted.
- Name - Monospaced type is used for code fragments, function prototypes and data type definitions.
- Function names are usually accompanied by parentheses (for example, `function()`).

## 2 Accessory

Workpath SDK defines API in `com.hp.workpath.api.accessory` package that can be called on Workpath enabled devices to authenticate with accessory from wakeup users or interact with USB accessory through events.

### 2.1 Authentication Process Initiation with Accessory

#### 2.1.1 Non-UI Initiated Authentication

Non-UI initiated authentication is triggered by some out-of-band user-initiated event (swiping a card, rebooting the device etc.) that is detected by the Accessory Agent implementation. An accessory receiver can be implemented in two ways: as a broadcast receiver or a service observer. Accessory Agent sample shows how to implement it as a broadcast receiver using `AbstractAuthenticationService`, while Accessory Service sample shows how to implement it as a service.

### 2.2 Accessory types for getting valid Accessory's context ID

To use an accessory, Workpath application must obtain accessory's context. The steps for getting accessory's context depend on accessory type whether accessory is "Shared" type or "Owned" type. Accessory sample shows how to use accessory based on types.

#### 2.2.1 Shared accessories

"Shared" accessories mean it's shared by multiple workpath applications. Workpath application must call `AccessoryService.reserveSharedAccessory()` to possess the valid context ID, then can access to the "Shared" accessory.

If reset is happened, the accessory's context is also destroyed, and the accessory is automatically closed.

#### 2.2.2 Owned accessories

"Owned" accessories mean the ability for an accessory to be owned by a single workpath application.

An owned accessory's context ID is transmitted from the device only to the owner Workpath application via an asynchronous event each time the accessory is discovered by the device (plug-and-play or at boot-up), and when a resend of the event is requested by the Workpath application.

## 3 Accessory flows

### 3.1 Registration

Accessory should be registered with vendor id and product id through installation of HPK file. Serial number is not mandatory parameter. If serial number is null, this registration applies to the set of all accessories whose vendor ID and product ID match exactly to the supplied values (for example, null is used as a wildcard for serial numbers).

### 3.2 Obtain valid context ID

If it's registered, valid accessory context id is returned when accessory is activated. Workpath application can request to resend the context id via `AccessoryService.reserveSharedAccessory()` for "Shared" accessory or `AccessoryService.resendOwnedAccessoryContext()` for "Owned" accessory. It depends on accessory registration type.

### 3.3 Open

Before interacting with the HID accessory, the workpath application must open it by calling `AccessoryService.open()` method.

### 3.4 Get specific accessory information

Workpath SDK provides the `AccessoryService.getInfo()` method to get the specific accessory information.

### 3.5 Get accessory event when active context is changed

When accessory context is changed, Workpath application can obtain event in two ways.

The first method is using `BroadcastReceiver` and the second method is using `AbstractAccessoryObserver.onContextChange()` method after register Accessory Service.

Please refer to section 6.2 Extends `BroadcastReceiver` and section 7 Accessory Service for more information.

### 3.6 Handle accessory reports

Workpath SDK passes all HID reports between the accessory and the Workpath application in raw form. Using specifications obtained from the accessory's manufacturer, the workpath

application is responsible for formatting all reports written to the accessory and for parsing all reports read from the accessory.

### **3.6.1 Asynchronously read reports**

To start the process of reading asynchronously from an open HID accessory, a Workpath application calls `startReading()` method. When this method is called, Workpath SDK prepares to receive events, then new input reports from the HID accessory are delivered to the workpath application through `onReceive()` method of `AbstractAccessoryObserver`. `onReceive()` events can be many per HID data, workpath application can distinguish events with the order of ordinals in `reportEventInfo`. Also, this ordinal helps to find the missed events. The application can call `stopReading()` to set the ordinal to zero and then prompt the user to use the prompt dialog to try again.

### **3.6.2 Synchronously read reports**

The Workpath application reads a report from an open HID accessory's control pipe by calling `readReport()`.

### **3.6.3 Write reports**

The Workpath application writes reports to an open HID accessory's control pipe by calling `writeReport()`. Reports are written to the HID accessory immediately. The written report ID is "0." (HID accessories that specify multiple report IDs are not recognized by the device.)

## **3.7 Close**

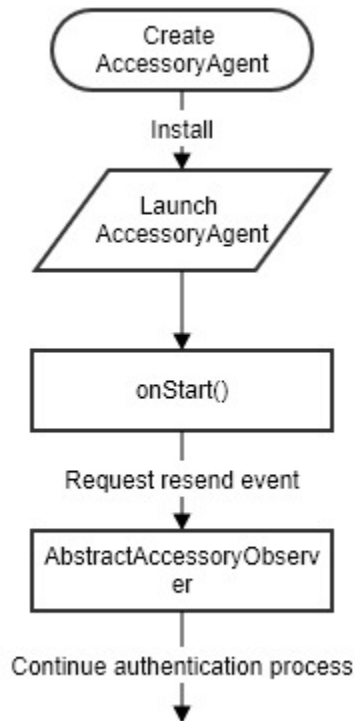
The Workpath application can call `close()` method to close accessory and clear context ID.

## **3.8 Unregistration**

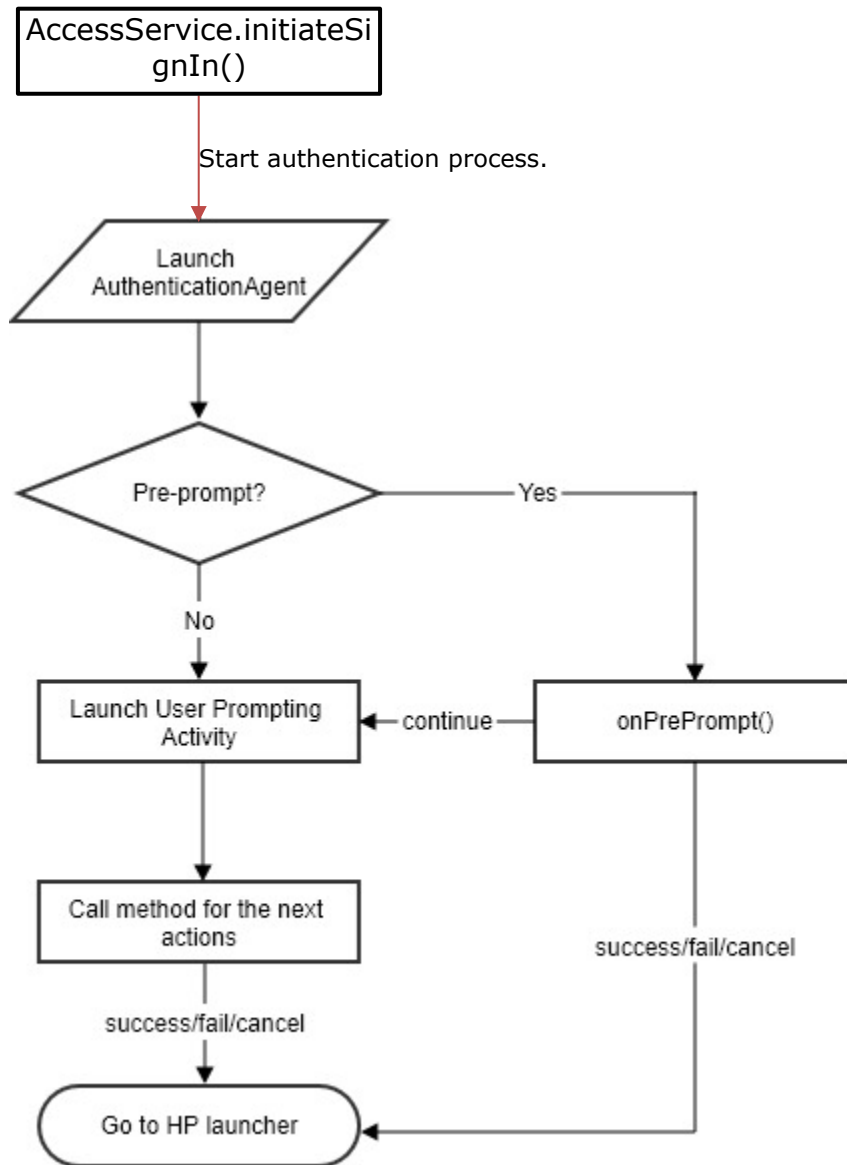
If application is deleted, registered accessory is unregistered. Walkup user will see the unregistered accessory error on front panel.

## 4 Authentication Process with accessory

When the Workpath initiates an accessory service to authenticate a device by front panel user, the Workpath SDK service will call the Accessory Agent to prepare observing accessory events.



When accessory event sends card data which is mapped to user, workpath application including accessory agent can request to start authentication process like select authentication method on front panel through `AccessService.initiateSignIn()`.



*Initiation Authentication Agent Process*

## 5 Create Accessory Agent for Authentication

To enable Accessory Agent for authentication, HPK file has to include Accessory information and Authentication Agent information. HPKTool helps to create Accessory Agent for authentication as follows:

1. Open HPKTool.

HPK Tool (Version 1.6.3) - HPK spec 1.5.0 (Schema v2.6)

File Actions Help

This page is for updating HPK

Install File: AccessoryAgentSample.apk [Select File]

UUID: 11111111-1111-1111-9990-111111111111 [Generate]

Name: Accessory Agent Sample

Vendor Name: HP

Platform version: 31.9 [Refresh]

Date: 20251111

Default Config: Default Configuration (Optional) [Select File] [Clear]

AuthAgent: AccessoryAgentSample [Add] [Delete]

Authorization: Authorization Name [Add] [Delete]

Home Screen App: ☐ Use Home Screen mode ☐ Set as default

Accessory [Add Table]

| Type  | Vendor ID (Decimal) | Product ID (Decimal) |   |    |
|-------|---------------------|----------------------|---|----|
| OWNED | 1008                | 69                   | i | 🗑️ |
| OWNED | 7590                | 272                  | i | 🗑️ |

WebService: Webservice Name [Add] [Delete] [Add Table]

*The dialog to create HPK file*

1. Click **Add**, then fill Accessory details. Workpath recommends to register information with type as “owned” and serial number as “Null” when integrate with authentication. Up to 10 accessory information can be registered.

The screenshot shows a dialog box titled "hp Accessory". It has a close button (X) in the top right corner. The "Type" field has two radio buttons: "Owned" (checked) and "Shared". The "Vendor ID (Decimal)" field contains the value "1008". The "Product ID (Decimal)" field contains the value "69". The "Serial Number" field has a text input with "Serial Number" and a checked "Null" radio button. At the bottom, there are "Save" and "Cancel" buttons.

An example for registering accessory

| Fields    |                      | Options  | Description                                                                                                                            |
|-----------|----------------------|----------|----------------------------------------------------------------------------------------------------------------------------------------|
| Accessory | Type                 | Required | Accessory type. “Owned” or “Shared”.                                                                                                   |
|           | Vendor ID (Decimal)  | Required | Accessory vendor ID                                                                                                                    |
|           | Product ID (Decimal) | Required | Accessory product ID                                                                                                                   |
|           | Serial Number        | Optional | Accessory serial number, NULL means to register application for all accessories which have the same vendor id and the same product id. |

Details of accessory for Workpath

2. Click **Add**, then fill Auth Agent details.

hp AuthAgent

Name (en-US)

AuthAgent Name

More

Description (en-US)

AuthAgent Description

More

Prompt Path

com.example.packagename.AuthAgentActivity

Pre-prompt

☐ Enable

Add

Cancel

An example for creating a AuthAgent

| Fields    |             | Options  | Description                                                                                                       |
|-----------|-------------|----------|-------------------------------------------------------------------------------------------------------------------|
| AuthAgent | Name        | Required | Name of AuthAgent that will be shown on sign-in method lists. Default is English and multi-language is supported. |
|           | Description | Required | Description of AuthAgent. Default is English and multi-language is supported.                                     |
|           | Prompt Path | Required | Prompt Path associated with AuthAgent. Value is defined by application. Expected value is activity path.          |
|           | Pre-prompt  | Optional | Flag to invoke pre-prompt service before launching Prompt.                                                        |

Details of AuthAgent

3. Install **HPK file** on a device.

## 6 Accessory Agent Sample on a device

The Workpath SDK provides AccessoryAgent sample to explain Accessory for authentication feature. The AccessoryAgent includes accessory status activity:



*An Accessory Agent Sample*

## 6.1 Extends **AbstractAuthenticationService**

If Workpath application extends `AbstractAuthenticationService` and adds the filter for overriding authentication events, `onPrePrompt()` event will be invoked by Workpath platform. Workpath application can call `AccessService.initiateSignIn()` for starting authentication process. Refer to `AccessoryAgentService.java` and `AndroidManifest.xml`.

```

1. public class AccessoryAgentService extends AbstractAuthenticationService {
2.
3.     public static final String TAG = MainActivity.TAG;
4.
5.     /* Background task for Workpath SDK API initialization */
6.     private InitializationTask mInitializationTask;
7.     private boolean isInitializedSDK = false;
8.
9.     @Override
10.    public void onCreate() {
11.        super.onCreate();
12.        initializedSDK();
13.    }
14.
15.    @Override
16.    public void onDestroy() {
17.        super.onDestroy();
18.
19.        if (isInitializedSDK) {
20.            mInitializationTask.cancel(true);
21.            mInitializationTask = null;
22.        }
23.    }
24.
25.    @Override
26.    protected void onSignIn(Principal principal) {
27.        Log.i(TAG, "Received sign in event: " + principal.getUsername());
28.    }
29.
30.    @Override
31.    protected void onSignOut() {
32.        Log.i(TAG, "Received sign out event");
33.    }
34.
35.    @Override
36.    protected void onPrePrompt() {
37.        Log.i(TAG, "onPrePrompt()");
38.        try {
39.            if (isInitializedSDK) {
40.                Result result = new Result();
41.                SignInAction signInAction = new SignInAction(ActionUtil.getAction(this)
42.                , null);
43.                AccessService.signIn(AccessoryAgentService.this, signInAction, getWindo
44.                wsData(), result);
45.                if (result != null && result.getCode() != Result.RESULT_OK) {
46.                    showLogResult("AccessService.signIn(): ", result);
47.                    SignInAction failAction = new SignInAction(SignInAction.Action.FAIL
48.                    , result.getCause());
49.                    AccessService.signIn(AccessoryAgentService.this, failAction, null,
50.                    null);
51.                }
52.            } else {

```

```

49.         Log.i(TAG, "isInitializedSDK is false");
50.     }
51. } catch (Exception e) {
52.     Log.e(TAG, "Please check the error: " + e.getMessage());
53. }
54. }
55.
56. private AuthenticationAttributes getWindowsData() throws Exception {
57.     String id = "Tester";
58.     String password = "password";
59.
60.     UserOverridesAttributes userOverridesAttributes = new UserOverridesAttributes.B
uilder()
61.         .addBccAddress(getString(R.string.bcc_address_email_01), getString(R.st
ring.bcc_address_name_01))
62.         .addBccAddress(getString(R.string.bcc_address_email_02), getString(R.st
ring.bcc_address_name_02))
63.         .addCcAddress(getString(R.string.cc_address_email_01), getString(R.stri
ng.cc_address_name_01))
64.         .addCcAddress(getString(R.string.cc_address_email_02), getString(R.stri
ng.cc_address_name_02))
65.         .setFrom(getString(R.string.from_address_email), getString(R.string.fro
m_address_name))
66.         .addToAddress(getString(R.string.to_address_email_01), getString(R.stri
ng.to_address_name_01))
67.         .addToAddress(getString(R.string.to_address_email_02), getString(R.stri
ng.to_address_name_02))
68.         .setMessage(getString(R.string.email_message))
69.         .setSubject(getString(R.string.email_subject))
70.         .setFaxBillingCode(getString(R.string.fax_billing_code))
71.         .setFaxCompanyName(getString(R.string.fax_company_name))
72.         .build();
73.
74.     UserPreferencesAttributes userPreferencesAttributes = new UserPreferencesAttrib
utes.Builder()
75.         .setAutoLaunchAppAccessPointId(getString(R.string.app_access_point_id))
76.         .setLanguageCode(getString(R.string.language_code))
77.         .build();
78.
79.     return new AuthenticationAttributes.WindowsBuilder()
80.         .setFullyQualifiedNames(getString(R.string.value_fully_qualified_name, i
d))
81.         .setDisplayName(id)
82.         .setPassword(password)
83.         .setUserDomain(getString(R.string.value_domain))
84.         .setUserEmail(getString(R.string.value_user_email, id))
85.         .setUserName(id)
86.         .setUserPrincipalName(id)
87.         .setHomeFolderPath(getString(R.string.value_home_folder_path))
88.         .addUserProperty(getString(R.string.value_user_property_key_01), getStr
ing(R.string.value_user_property_value_01))
89.         .addUserProperty(getString(R.string.value_user_property_key_02), getStr
ing(R.string.value_user_property_value_02))
90.         .setUserOverridesAttributes(userOverridesAttributes)
91.         .setUserPreferencesAttributes(userPreferencesAttributes)
92.         .build();
93. }
94.
95. private void initializedSDK() {
96.     try {

```

```

97.         if (!isInitializedSDK) {
98.             mInitializationTask = new InitializationTask(this, null);
99.             InitializationTask.InitStatus initStatus = mInitializationTask.execute(
100.         ).get();
101.             if (initStatus == InitializationTask.InitStatus.NO_ERROR) {
102.                 isInitializedSDK = true;
103.             } else {
104.                 Log.e(TAG, getString(R.string.sdk_support_missing));
105.             }
106.         } catch (Throwable t){
107.             Log.e(TAG, t.getMessage());
108.         }
109.     }

```

### *Extends AbstractAuthenticationService*

```

1. <service android:name=".service.AccessoryAgentService"
2.     android:permission="com.hp.jetadvantage.link.permission.SERVICES_PERMISSION">
3.     <intent-filter>
4.         <action android:name="com.hp.jetadvantage.link.api.action.AUTHENTICATION" />
5.     </intent-filter>
6. </service>

```

### *Filters AUTHENTICATION*

## 6.2 Extends BroadcastReceiver

If your application only uses OwnedAccessory, it can be implemented more easily by extending the BroadcastReceiver described in this chapter.

It can receive all data from accessories just by registering the event to receive in AndroidManifest.xml even if the application process is not running.

### 6.2.1 HID accessory data report in a broadcast intent

Starting with FS firmware version 5.7.1.1, this broadcast intent has been introduced. It handles all accessory types in the same manner, regardless of whether it's an HP card reader or a non-A card reader.

The following are the details of the broadcast intent:

- **com.hp.workpath.api.accessory.ACCESSORY\_CONTEXT\_CHANGE\_ACTION**
- **com.hp.workpath.api.accessory.ACCESSORY\_CHANGE\_ACTION**

| Detail                                      | Action Name                                                          | Key                              | Value              |
|---------------------------------------------|----------------------------------------------------------------------|----------------------------------|--------------------|
| Indicates the ready state of the accessory. | <b>com.hp.workpath.api.accessory.ACCESSORY_CONTEXT_CHANGE_ACTION</b> | <i>accessoryInfo</i>             | HIDAccessoryInfo   |
|                                             |                                                                      | <i>timestamp</i>                 | String             |
|                                             |                                                                      | <i>accessoryContextEventCode</i> | EventCode          |
|                                             |                                                                      | <i>accessoryContextId</i>        | String             |
| Indicates the data passed from accessory.   | <b>com.hp.workpath.api.accessory.ACCESSORY_CHANGE_ACTION</b>         | <i>accessoryInfo</i>             | HIDAccessoryInfo   |
|                                             |                                                                      | <i>hidReportEventInfo</i>        | HIDReportEventInfo |

#### ACCESSORY\_CONTEXT\_CHANGE\_ACTION

The ACCESSORY\_CONTEXT\_CHANGE\_ACTION intent is served when the accessory is attached, detached, or re-registered. When this intent is received, HIDAccessoryInfo can be obtained through the "accessoryInfo" key, the vendor ID and product ID of the connected device can be checked from that information and the card reader can be handled using the accessoryContextId received through the "accessoryContextId" key value.

Each information can be found in detail through the contents of the com.hp.workpath.api.accessory.hid package in the API Javadoc document.

#### ACCESSORY\_CHANGE\_ACTION

The ACCESSORY\_CHANGE\_ACTION broadcast intent is served when the accessory obtains data (for card readers, card data is passed when tagging a card). You can check which accessory received the information by looking at the HIDAccessoryInfo information passed through the "accessoryInfo" key, and you can check which information was passed by looking at the HIDReportEventInfo information passed through the "hidReportEventInfo" key.

Each information can be found in detail through the contents of the com.hp.workpath.api.accessory.hid package in the API Javadoc document.

```
1. package com.hp.workpath.sample.accessoryagentsample.receiver;
2.
3. import android.content.BroadcastReceiver;
4. import android.content.Context;
```

```

5. import android.content.Intent;
6. import android.os.Handler;
7. import android.text.TextUtils;
8. import android.util.Base64;
9. import android.util.Log;
10.
11. import com.hp.workpath.api.Result;
12. import com.hp.workpath.api.SdkUnsupportedException;
13. import com.hp.workpath.api.Workpath;
14. import com.hp.workpath.api.access.AccessService;
15. import com.hp.workpath.api.access.Principal;
16. import com.hp.workpath.api.accessory.RegistrationType;
17. import com.hp.workpath.api.accessory.hid.AccessoryService;
18. import com.hp.workpath.api.accessory.hid.EventCode;
19. import com.hp.workpath.api.accessory.hid.HIDAccessoryInfo;
20. import com.hp.workpath.api.accessory.hid.HIDInfo;
21. import com.hp.workpath.api.accessory.hid.HIDReport;
22. import com.hp.workpath.api.accessory.hid.HIDReportEventInfo;
23. import com.hp.workpath.sample.accessoryagentsample.Logger;
24. import com.hp.workpath.sample.accessoryagentsample.MainActivity;
25.
26. import java.lang.ref.WeakReference;
27. import java.util.List;
28.
29. public class AccessoryReceiver extends BroadcastReceiver {
30.     private static final String TAG = MainActivity.TAG + "Receiver";
31.     private static final String ACCESSORY_CONTEXT_CHANGE_ACTION =
"com.hp.workpath.api.accessory.ACCESSORY_CONTEXT_CHANGE_ACTION";
32.     private static final String ACCESSORY_CHANGE_ACTION =
"com.hp.workpath.api.accessory.ACCESSORY_CHANGE_ACTION";
33.     private WeakReference<Context> mContextRef;
34.     private Context appContext;
35.     private static boolean isInitializedSDK = false;
36.
37.     @Override
38.     public void onReceive(Context context, Intent intent) {
39.         appContext = context.getApplicationContext();
40.         this.mContextRef = new WeakReference<>(appContext);
41.
42.         initService(new Handler(), appContext);
43.
44.         if(ACCESSORY_CONTEXT_CHANGE_ACTION.equals(intent.getAction())
45.           || ACCESSORY_CHANGE_ACTION.equals(intent.getAction())){
46.             final String action = intent.getAction();
47.
48.             switch (action) {
49.                 case ACCESSORY_CONTEXT_CHANGE_ACTION:
50.                     onContextChange(intent);
51.                     break;
52.                 case ACCESSORY_CHANGE_ACTION:
53.                     onChange(intent);
54.                     break;
55.             }
56.         }
57.     }
58.
59.     private void initService(Handler handler, Context context) {
60.         try {
61.             Log.i(TAG, "initService Workpath.getInstance().initialize");
62.             if(!isInitializedSDK) {
63.                 Workpath.getInstance().initialize(mContextRef.get());
64.                 isInitializedSDK = true;
65.             }
66.         } catch (SdkUnsupportedException e) {
67.             Logger.showResult(null, e.getMessage());

```

```

68.     }
69. }
70.
71. private void openAccessory(String accessoryContextId) {
72.     Log.d(TAG, "Accessory opened: " + accessoryContextId);
73.     Result result = new Result();
74.     try{
75.         AccessoryService.open(appContext, accessoryContextId, result);
76.         Logger.showResult(null, "AccessoryService.open", result);
77.     } catch(Exception e) {
78.         Log.e(TAG,e.getMessage(),e);
79.     }
80.
81.     try{
82.         HIDInfo hidInfo = AccessoryService.getInfo(appContext,accessoryContextId, result);
83.         if (result.getCode() == Result.RESULT_OK) {
84.             Log.d(TAG, "HID feature report length : " + hidInfo.getFeatureReportLength());
85.             Log.d(TAG, "HID input report length : " + hidInfo.getInputReportLength());
86.             Log.d(TAG, "HID output report length : " + hidInfo.getOutputReportLength());
87.         }
88.     } catch(Exception e) {
89.         Log.e(TAG,e.getMessage(),e);
90.     }
91.
92.     try{
93.         AccessoryService.startReading(appContext, accessoryContextId, result);
94.         Logger.showResult(null, "AccessoryService.startReading", result);
95.     } catch(Exception e) {
96.         Log.e(TAG,e.getMessage(),e);
97.     }
98. }
99.
100.
101. private void initiateSignIn() {
102.     Result result = new Result();
103.     Log.d(TAG, "initiateSignIn");
104.     AccessService.initiateSignIn(appContext, result);
105.     Logger.showResult(null, "AccessService.initiateSignIn", result);
106. }
107.
108. private void signOut() {
109.     Result result = new Result();
110.     Log.d(TAG, "signOut");
111.     AccessService.signOut(appContext, result);
112.     Logger.showResult(null, "AccessService.signOut", result);
113. }
114.
115. private Principal getCurrentPrincipal() {
116.     Result result = new Result();
117.     Log.d(TAG, "getCurrentPrincipal");
118.     Principal principal = AccessService.getCurrentPrincipal(appContext, result);
119.     Logger.showResult(null, "AccessService.getCurrentPrincipal", result);
120.     if (result.getCode() == Result.RESULT_OK) {
121.         return principal;
122.     } else {
123.         return null;
124.     }
125. }
126.
127. public void onContextChange(Intent intent) {
128.     HIDAccessoryInfo hidAccessoryInfo = intent.getParcelableExtra("accessoryInfo");
129.     int vendorId = hidAccessoryInfo.getVendorId();
130.     int productId = hidAccessoryInfo.getProductId();
131.     String serialNumber = hidAccessoryInfo.getSerialNumber();
132.     RegistrationType registrationType = hidAccessoryInfo.getRegistrationType();

```

```

133.         String timeStamp = intent.getStringExtra("timestamp");
134.         EventCode eventCode = (EventCode)
intent.getSerializableExtra("accessoryContextEventCode");
135.         String accessoryContextId = intent.getStringExtra("accessoryContextId");
136.
137.         Log.i(TAG, "vendorId : "+vendorId);
138.         Log.i(TAG, "productId : "+productId);
139.         Log.i(TAG, "serialNumber : "+serialNumber);
140.         Log.i(TAG, "RegistrationType : "+registrationType);
141.         Log.i(TAG, "eventCode : "+eventCode);
142.         Log.i(TAG, "timeStamp : "+timeStamp);
143.         Log.i(TAG, "accessoryContextId : "+accessoryContextId);
144.
145.
146.         if(eventCode == EventCode.CONTEXT_CREATED || eventCode == EventCode.CONTEXT_RESENT){
147.             if (isInitializedSDK == false) {
148.                 Log.d(TAG, "The SDK has not been initialized yet.");
149.                 return;
150.             }
151.             if (!TextUtils.isEmpty(accessoryContextId)) {
152.                 openAccessory(accessoryContextId);
153.             }
154.         } else if(eventCode == EventCode.CONTEXT_REVOKED){
155.             Log.i(TAG, "Accessory context is revoked.");
156.         }
157.     }
158.
159.     public void onChange(Intent intent) {
160.         Log.i(TAG, "ACCESSORY_CHANGE_ACTION");
161.
162.
163.         HIDAccessoryInfo hidAccessoryInfo = intent.getParcelableExtra("accessoryInfo");
164.         int vendorId = hidAccessoryInfo.getVendorId();
165.         int productId = hidAccessoryInfo.getProductId();
166.         String serialNumber = hidAccessoryInfo.getSerialNumber();
167.
168.
169.         HIDReportEventInfo hidReportEventInfo =
intent.getParcelableExtra("hidReportEventInfo");
170.         long reportOrdinal = hidReportEventInfo.getOrdinal();
171.         String timeStamp = hidReportEventInfo.getTimestamp();
172.         List<HIDReport> reports = hidReportEventInfo.getReports();
173.
174.         Log.i(TAG, "vendorId : "+vendorId);
175.         Log.i(TAG, "productId : "+productId);
176.         Log.i(TAG, "serialNumber : "+serialNumber);
177.         Log.i(TAG, "ordinal : "+reportOrdinal);
178.         Log.i(TAG, "timeStamp : "+timeStamp);
179.         Log.i(TAG, "reportCount : "+reports.size());
180.
181.         for(int i=0; i<reports.size();i++){
182.             HIDReport hidReport = reports.get(i);
183.             byte[] data = hidReport.getData();
184.             Log.i(TAG, "report_"+(i+1)+" : "+ new String(Base64.encode(data,Base64.DEFAULT)));
185.             if(isHPCardReader(vendorId, productId) && new String(data).contains("\r")){
186.                 Principal principal = getCurrentPrincipal();
187.                 if (principal != null && !principal.isAuthenticated()) {
188.                     initiateSignIn();
189.                 } else {
190.                     signOut();
191.                 }
192.             }
193.         }
194.     }
195.

```

```

196.     private boolean isHPCardReader(int vendorId, int productId) {
197.         return vendorId == 1008 && productId == 69;
198.     }
199. }
200.

```

Extends **BroadcastReceiver**

```

1. <receiver android:name=".receiver.AccessoryReceiver"
2.         android:enabled="true"
3.         android:exported="true">
4.     <intent-filter>
5.         <action android:name="
6. com.hp.workpath.api.accessory.ACCESSORY_CONTEXT_CHANGE_ACTION"/>
7.         <action android:name="
8. com.hp.workpath.api.accessory.ACCESSORY_CHANGE_ACTION"/>
9.     </intent-filter>
10. </receiver>

```

*AndroidManifest.xml*

## 6.2.2 String-typed accessory data report in a broadcast intent

Starting with FS firmware version 5.6.0.2, this broadcast intent has been introduced. It manages HP card readers and non-HP card readers differently. Specifically for a HP card reader, the accessory report data is aggregated, processed together, and then delivered in a single broadcast intent. For a non-HP card reader, unprocessed report data is delivered in multiple broadcast intents. Sometimes it might deliver combined report data within a broadcast intent. The count of combined reports can be calculated using java API method `com.hp.workpath.api.accessory.hid.HIDInfo.getInputReportLength()`. If you are using only HP card readers and want a single broadcast, this broadcast intent is recommended.

The following are the details of the broadcast intent:

- **com.hp.workpath.action.ACCESSORY\_STATUS\_ACTION**
- **com.hp.workpath.action.ACCESSORY\_REPORT\_ACTION**

| Detail                                      | Action Name                                           | Key                                     | Value                                           |
|---------------------------------------------|-------------------------------------------------------|-----------------------------------------|-------------------------------------------------|
| Indicates the ready state of the accessory. | <b>com.hp.workpath.action.ACCESSORY_STATUS_ACTION</b> | <b>ACCESSORY_INFO_VENDOR_ID</b>         | Int                                             |
|                                             |                                                       | <b>ACCESSORY_INFO_PRODUCT_ID</b>        | Int                                             |
|                                             |                                                       | <b>ACCESSORY_INFO_SERIAL_NUMBER</b>     | String                                          |
|                                             |                                                       | <b>ACCESSORY_INFO_REGISTRATION_TYPE</b> | Int<br>0 (owned type)                           |
|                                             |                                                       | <b>ACCESSORY_INFO_EVENT_CODE</b>        | Int<br>0 (Created)<br>1 (Resent)<br>2 (Revoked) |
|                                             |                                                       | <b>ACCESSORY_INFO_CONTEXT_ID</b>        | String                                          |
|                                             |                                                       | <b>ACCESSORY_INFO_TIMESTAMP</b>         | String                                          |
| Indicates the data passed from accessory.   | <b>com.hp.workpath.action.ACCESSORY_REPORT_ACTION</b> | <b>ACCESSORY_INFO_VENDOR_ID</b>         | Int                                             |
|                                             |                                                       | <b>ACCESSORY_INFO_PRODUCT_ID</b>        | Int                                             |
|                                             |                                                       | <b>ACCESSORY_INFO_SERIAL_NUMBER</b>     | String                                          |
|                                             |                                                       | <b>ACCESSORY_REPORT_INFO_ORDINAL</b>    | Long                                            |
|                                             |                                                       | <b>ACCESSORY_REPORT_INFO_TIMESTAMP</b>  | String                                          |

|  |  |                                              |                             |
|--|--|----------------------------------------------|-----------------------------|
|  |  | <b>ACCESSORY_REPORT_INFO_DATA</b>            | String<br>(ASCII character) |
|  |  | <b>ACCESSORY_REPORT_INFO_BYTE_ARRAY_LIST</b> | ArrayList<byte[]>           |
|  |  | <b>ACCESSORY_REPORT_INFO_COUNT</b>           | Int                         |

## ACCESSORY\_STATUS\_ACTION

The ACCESSORY\_STATUS\_ACTION intent is served when the accessory is attached, detached, or re-registered. When this intent is received, you can determine the vendor ID and product ID of the connected device via the "ACCESSORY\_INFO\_VENDOR\_ID" and "ACCESSORY\_INFO\_PRODUCT\_ID" keys and process the card reader using the accessoryContextId received via the "ACCESSORY\_INFO\_CONTEXT\_ID" key.

## ACCESSORY\_REPORT\_ACTION

The ACCESSORY\_REPORT\_ACTION broadcast intent is served when the accessory obtains data (for card readers, card data is passed when tagging a card).

The vendor ID and product ID obtained through the "ACCESSORY\_INFO\_VENDOR\_ID" and "ACCESSORY\_INFO\_PRODUCT\_ID" keys can be used to determine which accessory received the information, and the String information passed through the "ACCESSORY\_REPORT\_INFO\_DATA" key can be used to determine which information was passed.

The String value for ACCESSORY\_REPORT\_INFO\_DATA can be converted into a raw data byte array with ASCII encoding. For example, if raw data is "0x61 0x62" in hexadecimal notation, the string would be "AB".

Additionally, in the case of HP card reader, the value "0x13" (CR), the last character of the data, is excluded from the string and transmitted.

Alternatively, you can use the "ACCESSORY\_REPORT\_INFO\_BYTE\_ARRAY\_LIST" key to get raw data.

"ACCESSORY\_REPORT\_INFO\_BYTE\_ARRAY\_LIST" and "ACCESSORY\_REPORT\_INFO\_COUNT" has been included since FS firmware version 5.7.1.1.

```

1. package com.hp.workpath.sample.accessoryagentsample.receiver;
2.
3. import android.content.BroadcastReceiver;
4. import android.content.Context;
5. import android.content.Intent;
6. import android.os.Handler;
7. import android.text.TextUtils;
8. import android.util.Log;
9.
10. import com.hp.workpath.api.Result;
11. import com.hp.workpath.api.SsdkUnsupportedException;
12. import com.hp.workpath.api.Workpath;
13. import com.hp.workpath.api.access.AccessService;
14. import com.hp.workpath.api.access.Principal;
15. import com.hp.workpath.api.accessory.hid.AccessoryService;
16. import com.hp.workpath.api.accessory.hid.HIDInfo;
17. import com.hp.workpath.sample.accessoryagentsample.Logger;
18. import com.hp.workpath.sample.accessoryagentsample.MainActivity;
19.
20. import java.lang.ref.WeakReference;
21.
22. public class AccessoryReceiver2 extends BroadcastReceiver {
23.     private static final String TAG = MainActivity.TAG + "Receiver2";

```

```

24.     private static final String ACCESSORY_STATUS_ACTION =
"com.hp.workpath.action.ACCESSORY_STATUS_ACTION";
25.     private static final String ACCESSORY_REPORT_ACTION =
"com.hp.workpath.action.ACCESSORY_REPORT_ACTION";
26.     private WeakReference<Context> mContextRef;
27.     private Context appContext;
28.     private static boolean isInitializedSDK = false;
29.
30.     @Override
31.     public void onReceive(Context context, Intent intent) {
32.         appContext = context.getApplicationContext();
33.         this.mContextRef = new WeakReference<>(appContext);
34.
35.         initService(new Handler(), appContext);
36.
37.         if(ACCESSORY_STATUS_ACTION.equals(intent.getAction())
38.           || ACCESSORY_REPORT_ACTION.equals(intent.getAction())){
39.             final String action = intent.getAction();
40.
41.             switch (action) {
42.                 case ACCESSORY_STATUS_ACTION:
43.                     onContextChange(intent);
44.                     break;
45.                 case ACCESSORY_REPORT_ACTION:
46.                     onChange(intent);
47.                     break;
48.             }
49.         }
50.     }
51.
52.     private void initService(Handler handler, Context context) {
53.         try {
54.             Log.i(TAG,"initService Workpath.getInstance().initialize");
55.             if(!isInitializedSDK) {
56.                 Workpath.getInstance().initialize(mContextRef.get());
57.                 isInitializedSDK = true;
58.             }
59.         } catch (SdkUnsupportedException e) {
60.             Logger.showResult(null,e.getMessage());
61.         }
62.     }
63.
64.     private void openAccessory(String accessoryContextId) {
65.         Log.d(TAG, "Accessory opened: " + accessoryContextId);
66.         Result result = new Result();
67.         try{
68.             AccessoryService.open(appContext, accessoryContextId, result);
69.             Logger.showResult(null, "AccessoryService.open", result);
70.         } catch(Exception e) {
71.             Log.e(TAG,e.getMessage(),e);
72.         }
73.
74.         try{
75.             HIDInfo hidInfo = AccessoryService.getInfo(appContext,accessoryContextId, result);
76.             if (result.getCode() == Result.RESULT_OK) {
77.                 Log.d(TAG, "HID feature report length : " + hidInfo.getFeatureReportLength());
78.                 Log.d(TAG, "HID input report length : " + hidInfo.getInputReportLength());
79.                 Log.d(TAG, "HID output report length : " + hidInfo.getOutputReportLength());
80.             }
81.         } catch(Exception e) {
82.             Log.e(TAG,e.getMessage(),e);
83.         }
84.
85.         try{
86.             AccessoryService.startReading(appContext, accessoryContextId, result);

```

```

87.         Logger.showResult(null, "AccessoryService.startReading", result);
88.     } catch(Exception e) {
89.         Log.e(TAG,e.getMessage(),e);
90.     }
91. }
92.
93. private void initiateSignIn() {
94.     Result result = new Result();
95.     Log.d(TAG, "initiateSignIn");
96.     AccessService.initiateSignIn(appContext, result);
97.     Logger.showResult(null, "AccessService.initiateSignIn", result);
98. }
99.
100. private void signOut() {
101.     Result result = new Result();
102.     Log.d(TAG, "signOut");
103.     AccessService.signOut(appContext, result);
104.     Logger.showResult(null, "AccessService.signOut", result);
105. }
106.
107. private Principal getCurrentPrincipal() {
108.     Result result = new Result();
109.     Log.d(TAG, "getCurrentPrincipal");
110.     Principal principal = AccessService.getCurrentPrincipal(appContext, result);
111.     Logger.showResult(null, "AccessService.getCurrentPrincipal", result);
112.     if (result.getCode() == Result.RESULT_OK) {
113.         return principal;
114.     } else {
115.         return null;
116.     }
117. }
118.
119.
120. public void onContextChange(Intent intent) {
121.     String timeStamp = intent.getStringExtra("ACCESSORY_INFO_TIMESTAMP");
122.     int eventCode = intent.getIntExtra("ACCESSORY_INFO_EVENT_CODE", -1);
123.     String accessoryContextId = intent.getStringExtra("ACCESSORY_INFO_CONTEXT_ID");
124.
125.     Log.i(TAG, "vendorId : "+intent.getIntExtra("ACCESSORY_INFO_VENDOR_ID", -1));
126.     Log.i(TAG, "productId : "+intent.getIntExtra("ACCESSORY_INFO_PRODUCT_ID", -1));
127.     Log.i(TAG, "serialNumber : "+intent.getStringExtra("ACCESSORY_INFO_SERIAL_NUMBER"));
128.     Log.i(TAG, "RegistrationType : "+intent.getIntExtra("ACCESSORY_INFO_REGISTRATION_TYPE",
-1));
129.     Log.i(TAG, "eventCode : "+eventCode);
130.     Log.i(TAG, "timeStamp : "+timeStamp);
131.
132.     if(eventCode == 0 || eventCode == 1){
133.         Log.i(TAG, "accessoryContextId: " + accessoryContextId);
134.         if (isInitializedSDK == false) {
135.             Log.d(TAG, "Initialized SDK is false");
136.             return;
137.         }
138.         if (!TextUtils.isEmpty(accessoryContextId)) {
139.             openAccessory(accessoryContextId);
140.         }
141.     } else if(eventCode == 2){
142.         Log.i(TAG, "Accessory context is revoked.");
143.     }
144. }
145.
146.
147. public void onChange(Intent intent) {
148.     Log.i(TAG, "onReceive()");
149.
150.     String timeStamp = intent.getStringExtra("ACCESSORY_INFO_TIMESTAMP");

```

```

151.         int eventCode = intent.getIntExtra("ACCESSORY_INFO_EVENT_CODE", -1);
152.         String accessoryContextId = intent.getStringExtra("ACCESSORY_INFO_CONTEXT_ID");
153.
154.         int vendorId = intent.getIntExtra("ACCESSORY_INFO_VENDOR_ID", -1);
155.         int productId = intent.getIntExtra("ACCESSORY_INFO_PRODUCT_ID", -1);
156.         String serialNumber = intent.getStringExtra("ACCESSORY_INFO_SERIAL_NUMBER");
157.         String reportData = intent.getStringExtra("ACCESSORY_REPORT_INFO_DATA");
158.
159.         Log.i(TAG, "accessoryContextId : "+accessoryContextId);
160.         Log.i(TAG, "vendorId : "+vendorId);
161.         Log.i(TAG, "productId : "+productId);
162.         Log.i(TAG, "serialNumber : "+serialNumber);
163.         Log.i(TAG, "eventCode : "+eventCode);
164.         Log.i(TAG, "timeStamp : "+timeStamp);
165.         Log.i(TAG, "reportData : "+reportData);
166.
167.         if(isHPCardReader(vendorId, productId)){
168.             Principal principal = getCurrentPrincipal();
169.             if (principal != null && !principal.isAuthenticated()) {
170.                 initiateSignIn();
171.             } else {
172.                 signOut();
173.             }
174.         }
175.     }
176.     private boolean isHPCardReader(int vendorId, int productId) {
177.         return vendorId == 1008 && productId == 69;
178.     }
179. }

```

*Extends BroadcastReceiver*

```

1.         <receiver android:name=".receiver.AccessoryReceiver2"
2.             android:enabled="true"
3.             android:exported="true">
4.             <intent-filter>
5.                 <action android:name="com.hp.workpath.action.ACCESSORY_STATUS_ACTION"/>
6.                 <action android:name="com.hp.workpath.action.ACCESSORY_REPORT_ACTION"/>
7.             </intent-filter>
8.         </receiver>

```

*AndroidManifest.xml*

## 6.3 Initiate authentication process

If the accessory's data is valid, Workpath application can start the authentication process calling `AccessService.initiateSignIn()`. This call for starting authentication agent makes the same behavior as the walkup user selecting the authentication menu on front panel.

```

1. private void initiateSignIn() {
2.     Result result = new Result();
3.     Log.d(TAG, "initiateSignIn");
4.     AccessService.initiateSignIn(getApplicationContext(), result);
5.     if (result == null || result.getCode() != Result.RESULT_OK) {
6.         showLogResult("AccessService.initiateSignIn(): ", result);
7.     }
8. }

```

*Calling AccessService.initiateSignIn method*

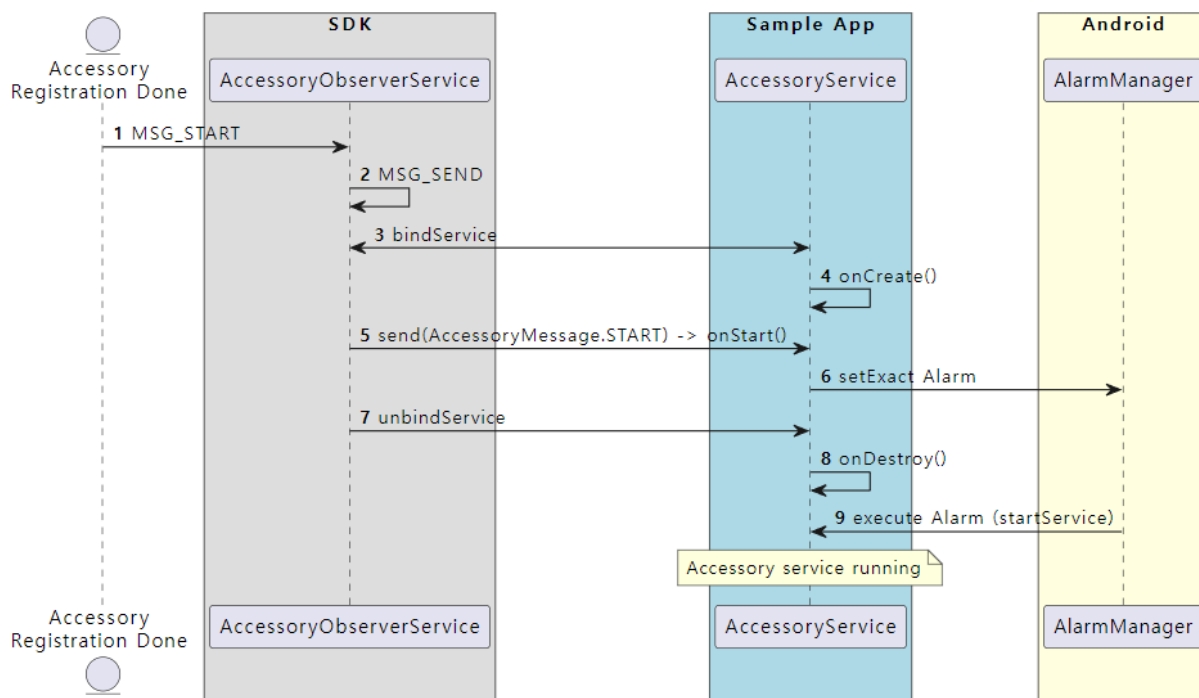
## 7 Accessory Service

Due to the Android policy change limiting services from Android 12 (FS firmware version 5.6.0.2), applications relying on AccessoryService would be affected if their Android targetSDKVersion is set to 31 or higher. Please refer the link below for more details.

<https://developer.android.com/about/versions/12/behavior-changes-12#foreground-service-launch-restrictions>

This section provides a sample guide on how to use accessories through a service for those who still prefer using the AccessoryService over a broadcast receiver.

### 7.1 Accessory Service Implementation



Accessory service should be implemented by inheriting from `AbstractAccessoryService`.

```
1. public class AccessoryService extends AbstractAccessoryService{}
```

### 7.2 Bind Service

Once the Accessory registration is successful, bind the service of a class that extends `AbstractAccessoryService` to notify the app and start the service.

This is achieved by extending a class that inherits from `AbstractAccessoryService`.

The purpose of this service is to notify the application about the successful registration and to start the service.

## 7.3 Alarm Settings

Utilize AlarmManager to set an alarm for service execution.

- Set uses-permission in AndroidManifest for using exact alarm.

```
1. <uses-permission android:name="android.permission.SCHEDULE_EXACT_ALARM" />
```

## 7.4 onStart()

When the service is initiated, onStart() method is called.

This method indicates that the Accessory is ready to use.

At this point, the app should initiate a Service that continuously receives data from the Accessory through the AccessoryObserver.

Starting from Android 12, calling one service directly from another service in the background is not allowed.

Therefore, it is essential to use the AlarmManager to initiate the necessary service.

The AlarmManager ensures that your service runs at the scheduled intervals without violating Android 12's background execution restrictions.

- Register the service to be executed in alarm manager.

```
1. @Override
2.     protected void onStart() {
3.         Log.d(TAG, "onStart");
4.
5.         Intent intent = new Intent(getApplicationContext(), getClass());
6.         PendingIntent pi = PendingIntent.getForegroundService(getApplicationContext(), 1,
intent, PendingIntent.FLAG_IMMUTABLE);
7.         AlarmManager am = (AlarmManager)
getApplicationContext().getSystemService(Context.ALARM_SERVICE);
8.         am.setExact(AlarmManager.ELAPSED_REALTIME_WAKEUP, SystemClock.elapsedRealtime(), pi);
9.     }
10.
```

## 7.5 Unbind Service

The AbstractAccessoryService role is complete once it receives a signal that the accessory is ready for use.

## 7.6 Execute accessory service through AlarmManager

Subsequently, when the Service registered with AlarmManager starts, you can proceed to register the AccessoryObserver and receive data as before.

More information on how to register an AccessoryObserver and process the data received can be found in the Accessory Overview document.

## 8 Legal notice

The information contained in this documentation is subject to change without notice.

HP Inc. makes no warranty of any kind with regard to this material, including, but not limited to, the implied warranties of merchantability and fitness for a particular purpose. HP Inc. shall not be liable for errors contained herein or for incidental or consequential damages in connection with the furnishing, performance, or use of this material.

This document contains proprietary information that is protected by copyright. All rights are reserved. No part of this documentation may be transformed, reproduced, or translated to another language without prior.

© Copyright 2018 HP Development Company, L.P.

## 9 Support

The HP Solution Business Program (SBP) support staff has established a procedure for submitting technical problems to the Incident Call Center (ICC). All SBP members must be registered with the Solution Programs Portal (SPP) to access technical support and other feedback or reporting activities.

Log on to the portal at <<http://www.hp.com/go/solutions>> to submit your issue to technical support. After you submit the issue, a report will be generated for the Incident Call Center. A Technical Support representative will be assigned to validate the incident and to work on a fix. HP will contact you for further information regarding the incident as quickly as possible.